

KESHAV KUMAR JHA

Python Backend Developer | Django Developer | REST API Development | AI/LLM Integration

Uttam Nagar, New Delhi | +91 9667131795 | kjkeshav0698@gmail.com

[Portfolio](#) | [LinkedIn](#) | [GitHub](#)

Professional Project Summary

This portfolio showcases my software development projects and technical learning repositories, demonstrating hands-on experience in Python, Django, PostgreSQL, REST API development, frontend development, database design, and AI-assisted software engineering. It features **LifeChronicle** as my flagship specification-driven software engineering project, along with multiple backend and full-stack applications, a personal portfolio website, and dedicated learning repositories covering Core Python, Data Structures & Algorithms, and problem solving, reflecting my continuous learning, practical implementation, and commitment to modern software development.

Software Project Portfolio

1. LifeChronicle (Flagship Project – In Progress)

Technologies

Python • Django • PostgreSQL • HTML5 • CSS3 • Bootstrap • JavaScript • Git • GitHub

Project Overview:

LifeChronicle is an enterprise-scale digital life management platform engineered using a Specification-Driven Software Development (SDSD) approach. The project focuses on building a scalable, modular, and production-ready application while adhering strictly to the full Software Development Lifecycle (SDLC) best practices.

Key Engineering Highlights:

Specification-Driven Architecture: Architecting the entire system based on 19 comprehensive technical documents, including Software Requirements Specification (SRS), domain business rules, system architecture, database design schemas, data validation strategies, and technical documentation.

Database & Modular Design: Engineering a scalable PostgreSQL backend featuring 30+ relational database models structured across 9 modular Django applications to maintain high cohesion and low coupling.

REST API & Security: Developing 30+ RESTful API endpoints using Django REST Framework (DRF) with secure JWT authentication, dynamic URL routing, granular permissions, and structured API documentation.

Parallel Application Development: Simultaneously implementing frontend and backend modules for authentication, profile management, event tracking, media asset storage, location services, categories, and administrative panels.

Enterprise Features: Completed full master data management and native 7-language dynamic multilingual support (i18n) alongside responsive web user interface modules.

AI/LLM Integration: Engineering workflows to integrate AI/LLM-powered features (ChatGPT, Claude, Gemini APIs) for automated user data enrichment, intelligent insight generation, and contextual analytics.

Software Engineering Standards: Applying Git-based version control, modular code architecture, testability, maintenance strategies, and production-ready cloud deployment pipelines (Render) throughout the development lifecycle.

Links: [Documentation & Development Progress](#)

2. DevForge (Full-Stack Development & Productivity Platform – Completed)

Technologies

Python • Django • Django REST Framework • PostgreSQL • HTML5 • CSS3 • Bootstrap • JavaScript • Google OAuth 2.0 • Gmail API • SMTP • OpenAI API • Gemini API • Leaflet • Git • GitHub • Render

Project Overview:

DevForge is a full-stack personal development and productivity platform built with a modular Django architecture. It integrates authentication, email communication, location management, AI-assisted workflows, file management, reporting, and analytics into a unified application.

Key Engineering Highlights:

Modular Architecture: Built across **6 Django applications** — Accounts, Emails, Location, AI, Files, and Reports — with clear separation of responsibilities.

Database Design: Implemented **5 core relational models** for profiles, emails, locations, AI interactions, and files using PostgreSQL.

Backend Development: Developed **10+ Django views** covering authentication, dashboard, email, AI, location, file, reporting, and analytics workflows.

REST API Development: Implemented **3 Django REST Framework API endpoints** to establish the application's RESTful API layer.

Authentication & Profile Management: Integrated Google OAuth 2.0 with Django Allauth, including automatic profile creation, Google account metadata, profile information, profile photo, email verification, and Gmail connectivity.

Gmail & Email Integration: Integrated Gmail with the gmail.send scope for sending emails through connected Gmail accounts, along with SMTP system emails, welcome emails, CC/BCC, attachments, status tracking, and AI-assisted email workflows.

AI/LLM Integration: Integrated **OpenAI and Gemini APIs** for AI-assisted email generation/improvement and intelligent location-description generation.

Location & Mapping: Implemented visited/planned locations with addresses, coordinates, interactive Leaflet maps, and AI-assisted descriptions.

File Management: Added centralized file handling for **PDF, DOC, DOCX, and TXT** with validation, metadata, size restrictions, and email/AI associations.

Reports & Analytics: Built reporting functionality that reads directly from existing activity models, with structured presentation and **10-record pagination**.

Cloud Deployment: Deployed the application on **Render** with PostgreSQL, environment-based secrets, WhiteNoise static-file handling, SMTP, and Google OAuth configuration.

Engineering Practices: Applied Git-based version control, modular architecture, relational database design, REST APIs, authentication security, validation, reusable services, AI integration, and cloud deployment practices.

Links: [Live DevForge](#) • [GitHub](#)

3. Staff Management System

Technologies

Python • Django • PostgreSQL • Django ORM • HTML5 • CSS3 • Bootstrap

- Developed a modular Django MVT staff management application (1 app, 1 core model) backed by PostgreSQL to manage full CRUD workflows, database migrations, and URL routing.

- Built clean, mobile-responsive user interfaces using Bootstrap 5, Django template inheritance, and optimized ORM queries.

Links: [GitHub](#) | [Live Demo](#)

3. Exam Center Management System

Technologies

- Python • Django • PostgreSQL • Django ORM • Django Forms • HTML5 • CSS3 • Bootstrap
- Architected a 5-page Django exam center administration tool (1 app, 1 database model) backed by PostgreSQL to manage examination center records.
 - Implemented Django ModelForms for server-side validation, ORM-based database operations, clean routing, and structured template inheritance.

Links: [GitHub](#) | [Live Demo](#)

4. Registration & Login System

Technologies

- HTML5 • CSS3 • Bootstrap • JavaScript • JSON • Local Storage
- Engineered a 6-page responsive authentication portal featuring registration, login, password recovery, profile photo upload, session logout, dynamic dashboard, and client-side validation.
 - Implemented client-side data persistence using JSON schemas and browser LocalStorage for offline state management.

Links: [GitHub](#) | [Live Demo](#)

5. Personal Portfolio Website

Technologies

- HTML5 • CSS3 • JavaScript • GitHub Pages
- Designed and deployed a single-page responsive personal portfolio on GitHub Pages showcasing technical projects, GitHub repositories, and developer credentials.
 - Leveraged AI-assisted workflows (ChatGPT, Gemini, Claude) for rapid code generation, UI refinement, and cross-browser debugging.

Links: [GitHub](#) | [Live Portfolio](#)

Technical Learning Repositories

Python Core : [GitHub](#)

Core Python, OOP, Functions, Modules, File Handling, Exception Handling, and practical programming exercises.

Data Structures & Algorithms : [GitHub](#)

Arrays, Strings, Linked Lists, Stacks, Queues, Trees, Graphs, Searching, Sorting, and interview-oriented implementations.

Python Problem Solving : [GitHub](#)

Coding exercises covering strings, arrays, mathematical logic, patterns, loops, and interview-focused programming questions.